

5 Hard Questions for GenAI Developer Role

Question 1: Advanced RAG System Design with Multi-Modal Context

Scenario: You're building a RAG system for a financial services company that needs to answer questions using:

- PDF financial reports (text + tables)
- Quarterly earnings call transcripts (audio → text)
- Stock price charts and financial graphs (images)
- Real-time market data feeds
- Historical Q&A pairs from analyst calls

The system must:

- Handle queries like "How did revenue growth compare to operating margin expansion in Q3, and what does the chart on page 15 suggest about future trends?"
- Provide source citations with page numbers and timestamps
- Work with documents updated daily
- Serve 1000+ concurrent users with <2 second latency

Design this system end-to-end. Address:

Part A: Architecture & Data Pipeline

1. How would you structure your chunking strategy for documents with mixed content (text, tables, charts)?
2. How do you handle the multi-modal nature (text, images, audio)?
3. Describe your embedding strategy - single model or multiple specialized models?
4. How do you maintain temporal consistency with daily updates?

Part B: Retrieval & Accuracy

5. Design a retrieval pipeline that can find relevant information across different modalities
6. How do you ensure table data is correctly interpreted and retrieved?
7. How would you implement hybrid search (semantic + keyword + structured data)?
8. Describe your re-ranking strategy to improve precision

Part C: Performance & Scale

9. How do you achieve <2 second latency with 1000+ concurrent users?
10. Describe your caching strategy across different layers
11. How do you handle the trade-off between comprehensive retrieval and speed?
12. What monitoring and fallback mechanisms would you implement?

Part D: Quality & Governance

13. How do you prevent hallucinations when the model doesn't find relevant context?
 14. How do you implement source citation that's verifiable?
 15. How do you handle contradictory information across different source documents?
 16. What testing strategy would you use before production deployment?
-

Expected Answer Should Demonstrate:

Architecture Excellence:

- Multi-stage retrieval (fast filtering → precise ranking)
- Separate pipelines for different document types
- Document preprocessing: PDFs → text extraction with layout preservation, table extraction (Camelot/Tabula), image extraction
- Audio transcription with speaker diarization and timestamps
- Metadata extraction and enrichment

Embedding Strategy:

- Specialized embeddings: ColBERT for long documents, CLIP for images

- Hybrid approach: dense (semantic) + sparse (BM25/SPLADE) embeddings
- Table-specific embeddings or structured representation
- Consideration of embedding dimension trade-offs (512 vs 1536 vs 3072)

Advanced Retrieval:

- Query decomposition for complex multi-part questions
- Graph-based retrieval linking related concepts
- Temporal filtering for recent data prioritization
- Cross-modal retrieval (text query → image results)
- Parent-child chunking for context preservation

Performance Optimization:

- Multi-level caching: embedding cache, retrieval results cache, LLM response cache with semantic similarity
- Vector DB index optimization (HNSW with optimized parameters)
- Async processing and parallel retrieval
- Model quantization and batch inference
- CDN for static embeddings
- Request deduplication and query coalescing

Quality Control:

- Confidence scoring for retrieved chunks
- Ensemble retrieval with voting mechanism
- Explicit "I don't know" responses when confidence is low
- Source attribution with direct quotes and page numbers
- Retrieval evaluation metrics (MRR, NDCG, Hit Rate)
- Continuous A/B testing

Production Concerns:

- Incremental updates without full re-indexing
- Blue-green deployment for embedding model updates

- Circuit breakers and graceful degradation
 - Cost optimization (caching expensive LLM calls)
 - Compliance and audit logging
-

Question 2: LLM Fine-tuning vs Prompt Engineering Decision Framework

Scenario: Your company has a customer support chatbot that needs to:

- Understand domain-specific terminology (proprietary product names, technical jargon)
- Follow specific response formats and tone guidelines
- Handle 20 different product categories with unique Q&A patterns
- Maintain conversation context across multiple turns
- Currently uses GPT-4 with RAG, but costs are \$50K/month
- Response quality is 75% satisfactory based on user feedback

Your task:

Part A: Analysis & Strategy

1. Create a decision framework: when to use prompt engineering vs RAG vs fine-tuning vs all three?
2. For this specific scenario, justify your approach with cost-benefit analysis
3. If you choose fine-tuning, which base model would you select and why? (Consider: Llama 3, Mistral, GPT-3.5, Claude)

Part B: Fine-tuning Implementation (if chosen)

4. Describe your data collection and preparation strategy for fine-tuning
5. How many examples do you need? What quality criteria?
6. What fine-tuning approach: full fine-tuning vs LoRA vs QLoRA? Why?
7. How do you prevent catastrophic forgetting?
8. What evaluation methodology would you use?

Part C: Hybrid Approach Design

9. Design a system that combines prompt engineering, RAG, and fine-tuning optimally
10. How do you route queries to different strategies?
11. Describe your prompt template structure for consistency

Part D: Optimization & Measurement

12. How would you reduce the \$50K/month cost while maintaining quality?
 13. Define success metrics beyond user satisfaction
 14. How do you continuously improve the system?
-

Expected Answer Should Demonstrate:

Decision Framework:

- **Prompt Engineering:** For general behavior, formatting, reasoning patterns. Low cost, fast iteration. Use first.
- **RAG:** For factual knowledge, dynamic content, product-specific info. Moderate cost, scalable knowledge.
- **Fine-tuning:** For domain language, consistent tone/style, specialized reasoning patterns, cost reduction at scale.

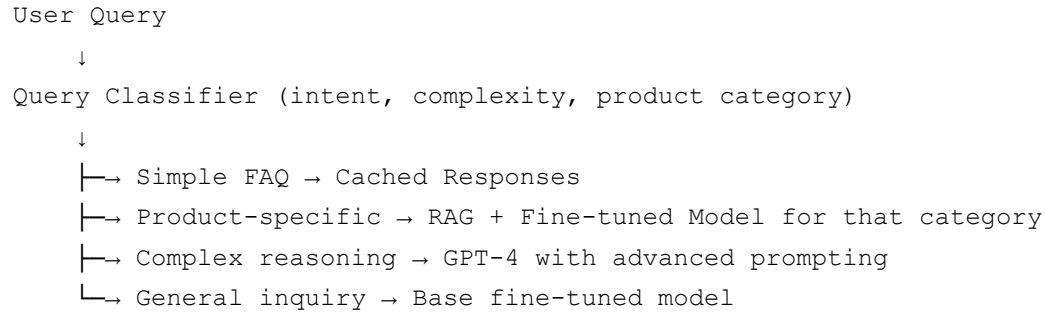
Strategic Thinking:

- ROI calculation: Fine-tuning upfront cost vs reduced inference cost
- Product-specific fine-tuned models vs single model with sophisticated prompting
- Risk analysis: model drift, maintenance overhead, deployment complexity

Technical Depth on Fine-tuning:

- Data quality over quantity: 500-1000 high-quality examples > 10K poor ones
- Data format: instruction-response pairs with system context
- LoRA/QLoRA for efficiency (4-8 bit quantization, low-rank adaptation)
- Hyperparameter tuning: learning rate, batch size, epochs, rank (r) for LoRA
- Validation on held-out product categories to test generalization

Hybrid Architecture:



Cost Optimization:

- Aggressive caching (semantic similarity cache)
- Fine-tuned smaller model (Mistral 7B) for 80% of queries
- GPT-4 only for complex edge cases
- Prompt compression techniques
- Self-hosted open-source models for non-sensitive queries
- Reduce token usage through better prompting

Measurement:

- User satisfaction (thumbs up/down)
- Resolution rate (issue solved in single interaction)
- Escalation rate (transferred to human)
- Response accuracy (validated against ground truth)
- Cost per conversation
- Latency metrics

Question 3: Production LLM Hallucination Mitigation

Scenario: Your production RAG system for medical literature review is experiencing hallucination issues:

- 5% of responses contain fabricated citations
- Occasionally makes up non-existent clinical studies

- Sometimes misattributes findings to wrong papers
- Combines facts from different sources incorrectly
- Users (medical researchers) are losing trust

This is critical because wrong information could influence medical decisions.

Part A: Root Cause Analysis

1. Design a systematic approach to identify when and why hallucinations occur
2. How do you differentiate between: (a) hallucination, (b) incorrect retrieval, (c) misinterpretation of source?
3. Create a taxonomy of hallucination types you'd track

Part B: Prevention Mechanisms

4. Design multiple layers of hallucination prevention in the pipeline
5. How do you modify the RAG retrieval to reduce hallucination risk?
6. What prompt engineering techniques specifically combat hallucinations?
7. Should you fine-tune the LLM? If so, how would you create anti-hallucination training data?

Part C: Detection & Guardrails

8. Design a real-time hallucination detection system
9. How do you validate citations without expensive API calls?
10. Create a confidence scoring mechanism for responses
11. When should the system refuse to answer vs provide low-confidence response?

Part D: Testing & Validation

12. Design a test suite to catch hallucinations before production
13. How do you create adversarial test cases?
14. What does your continuous monitoring system look like?

Expected Answer Should Demonstrate:

Root Cause Analysis:

- Logging framework capturing: query, retrieved chunks, chunk scores, LLM response, citations
- A/B testing different retrieval strategies
- Analysis of hallucination patterns: specific topics? certain query types? retrieval score thresholds?
- User feedback loop for reported issues

Hallucination Taxonomy:

- **Factual hallucinations:** Made-up facts
- **Citation hallucinations:** Non-existent sources
- **Attribution errors:** Correct fact, wrong source
- **Synthesis errors:** Incorrectly combining multiple sources
- **Temporal errors:** Outdated information presented as current

Prevention Layers:

Layer 1 - Retrieval Quality:

- Increase similarity threshold (only high-confidence chunks)
- Require minimum number of supporting chunks
- Cross-reference retrieval with multiple embedding models
- Implement query decomposition for complex questions
- Use metadata filtering (publication date, journal reputation)

Layer 2 - Prompt Engineering:

CRITICAL RULES:

1. ONLY use information from provided context
2. If context insufficient, say "I cannot find information about X in the provide"
3. NEVER make up citations - only cite if document ID provided
4. Quote directly when making specific claims
5. Distinguish between "the document states" vs "this suggests"
6. If sources conflict, explicitly state the conflict

Layer 3 - Response Validation:

- Extract all citations from response
- Verify each citation exists in retrieved documents
- Check if quoted text actually appears in source
- NLI (Natural Language Inference) model to verify claim-evidence alignment
- Named Entity Recognition to catch fabricated entities

Layer 4 - Confidence Scoring:

```
confidence_score = weighted_average([
    retrieval_score,          # How well chunks match query
    citation_coverage,        # % of claims with citations
    semantic_consistency,     # NLI score between claim and evidence
    retrieval_agreement,      # Multiple chunks support same fact
    llm_token_probability     # If accessible
])

if confidence_score < threshold:
    return "Low confidence - please verify with original sources"
```

Detection Mechanisms:

1. Citation Validator:

- Regex extract citations: [Author, Year] or footnote numbers
- Check against retrieved document metadata
- Flag if citation format present but not in sources

2. Claim Verification:

- Use smaller LLM to check if claim supported by retrieved text
- Semantic similarity between response sentences and source sentences
- Entailment model: does source entail the claim?

3. Consistency Checker:

- Generate same response with different LLM
- Compare outputs - agreement indicates higher confidence
- Use multiple retrieval strategies, check consistency

Testing Strategy:

```

test_cases = [
    # Positive tests
    {"query": "...", "expected_sources": [...], "must_cite": True},

    # Negative tests - should decline to answer
    {"query": "What did study X conclude?", "no_relevant_docs": True, "should_dec

    # Adversarial tests
    {"query": "Tell me about the 2023 Johnson study on...", "non_existent_study":

    # Ambiguity tests
    {"query": "Does treatment A work?", "conflicting_sources": True, "must_acknow
]

```

Production Monitoring:

- Track citation failure rate
- User feedback on accuracy
- Automated fact-checking against knowledge base
- Citation format anomaly detection
- Regular audit sample manual review
- A/B test improvements

When to Refuse:

- Retrieval score below threshold
- No citations found in response
- Confidence score below threshold
- Conflicting high-confidence sources
- Query outside domain scope

Question 4: Context Window Management & Long-Context Reasoning

Scenario: You're building a legal contract analysis system where:

- Contracts are 50-200 pages (50K-200K tokens)

- Users ask questions requiring synthesis across entire document
- Example: "Are there any conflicts between the liability terms in Section 3 and the indemnification clause in Section 12?"
- Questions may reference multiple contracts simultaneously
- You have access to Claude (200K context) and GPT-4 Turbo (128K context)

Part A: Architecture Decisions

1. Compare approaches: (a) use full context window, (b) RAG with retrieval, (c) hybrid approach. When is each appropriate?
2. If using long context: how do you optimize token usage? Contracts have boilerplate, tables, signatures that may not be needed.
3. If using RAG: how do you avoid missing connections between distant sections?
4. Design a system that intelligently decides which approach for each query.

Part B: Advanced Context Management

5. Implement "needle in haystack" test - how do you ensure model finds relevant clauses even at 150K tokens?
6. Design a hierarchical summarization system: chunk summaries → section summaries → document summary. How do you use this?
7. How do you handle cross-document queries (referencing multiple 100K token contracts)?
8. Create a strategy for handling follow-up questions with conversation history.

Part C: Cost & Performance Optimization

9. Long context is expensive (\$20-40 per 1M tokens). How do you optimize costs while maintaining accuracy?
10. Design a caching strategy for contract analysis (contracts don't change, but questions vary).
11. How do you reduce latency when processing 200K token contexts?

Part D: Quality & Evaluation

12. How do you evaluate whether full context is actually being used effectively vs just recent tokens?

13. Design test cases that specifically test long-range dependencies.
 14. What metrics indicate the model is "losing" information in the middle?
-

Expected Answer Should Demonstrate:

Sophisticated Analysis:

When to use Full Context:

- Simple queries needing document overview
- When sections are tightly coupled
- Budget allows (\$0.01-0.02 per query with 100K context)
- Latency not critical (can take 10-30s)

When to use RAG:

- Well-defined questions with clear intent
- Cost-sensitive applications
- Need for speed (<2s response)
- Query clearly scoped to specific sections

Hybrid Approach (Best):

1. Query Analysis:
 - Classify query complexity (simple, medium, complex)
 - Identify referenced sections
 - Determine if cross-document
2. Initial Retrieval (Fast):
 - Semantic search for top-k relevant sections
 - Score: how concentrated is relevance? (all in one section vs spread)
3. Decision Logic:

```
if relevance_spread_score > threshold:
    use_full_context() # Need whole document
elif retrieval_confidence > high_threshold:
    use_rag_only()     # Clear answer location
else:
    use_hybrid()       # Retrieve + surrounding context
```

Advanced Techniques:

1. Intelligent Context Pruning:

```
def optimize_context(document, query, max_tokens):  
    # Remove standard boilerplate  
    doc = remove_boilerplate(document)  
  
    # Identify relevant sections via semantic search  
    relevant_sections = semantic_search(query, doc, top_k=5)  
  
    # Expand with surrounding context  
    expanded_sections = expand_with_neighbors(relevant_sections)  
  
    # Add document structure (ToC, section headers)  
    structure = extract_structure(doc)  
  
    # Compress if still too large  
    if estimate_tokens(expanded_sections) > max_tokens:  
        compressed = summarize_less_relevant_sections(expanded_sections)  
  
    # Construct optimized context  
    final_context = structure + compressed + relevant_sections  
  
    return final_context
```

2. Hierarchical Summarization:

```
Level 1: Paragraph summaries (reduce 80%)  
Level 2: Section summaries (reduce 90%)  
Level 3: Document summary (reduce 95%)
```

Query Router:

- Simple question → Level 3 summary + RAG on original
- Medium question → Level 2 summaries + relevant full sections
- Complex cross-section → Level 1 + full context of relevant sections

3. Cross-Document Strategy:

- Pre-compute document embeddings at multiple levels
- Use document summary for initial filtering
- Load only relevant full documents into context
- Create unified reference system (Doc1-Sec3-Para2)

4. Conversation Context Management:

```
conversation_tokens = user_history + assistant_history
document_tokens = relevant_sections

if (conversation_tokens + document_tokens) > context_limit:
    # Option 1: Summarize conversation history
    compressed_history = summarize_conversation(user_history[-10:])

    # Option 2: Keep only relevant conversation turns
    relevant_history = filter_relevant_turns(user_history, current_query)

    # Option 3: Store document embeddings, re-retrieve fresh
    document_sections = fresh_retrieval(current_query)
```

Cost Optimization:

Caching Strategy:

```
Cache Layer 1: Document embeddings (permanent)
Cache Layer 2: Section summaries (permanent)
Cache Layer 3: Full document in vector DB (permanent)
Cache Layer 4: Q&A pairs with semantic similarity (30 days)
Cache Layer 5: Document analysis results (e.g., "key terms", "parties") (permanent)

Estimated cost reduction: 70-80% through caching
```

Token Optimization:

- Remove white space, formatting artifacts
- Compress tables to key information
- Use pointer-based references instead of repeating text
- Async processing: analyze document once, cache results

Quality Evaluation:

Needle in Haystack Tests:

```
test_cases = [
    # Place critical clause at token position 10K, 50K, 100K, 150K
    {"needle_position": [10000, 50000, 100000, 150000],
     "query": "What is the liability cap?",
     "answer": "$5 million", # Embedded at various positions
     "must_find": True},
```

```
# Multi-hop reasoning
{"needles": [("Section 3", "token_10K"), ("Section 12", "token_120K")],
 "query": "Do Section 3 and 12 conflict?",
 "requires_both": True}
]
```

Middle Context Loss Detection:

- Place test facts at start, middle, end
- Track retrieval success rate by position
- Use attention visualization if available
- Compare GPT-4 vs Claude (Claude better at long context)

Metrics:

- Position-aware accuracy (start/middle/end)
 - Cross-reference accuracy (did it connect A to B?)
 - Cost per query (token usage)
 - Latency percentiles (p50, p95, p99)
 - Cache hit rate
-

Question 5: Multi-Agent LLM System with Tool Use

Scenario: Design an autonomous research assistant that:

- Takes a research question (e.g., "What are the latest advances in quantum computing for drug discovery?")
- Searches academic papers, news, patents, and preprints
- Reads and synthesizes 20-50 documents
- Creates a comprehensive research report with citations
- Identifies key researchers and institutions
- Suggests follow-up research directions

The system must be autonomous, accurate, and complete the task in 10-15 minutes.

Part A: Agent Architecture

1. Design a multi-agent system: what agents do you need? How do they coordinate?
2. When do you use ReAct vs Chain-of-Thought vs Tree of Thoughts vs agentic workflows?
3. How do you prevent infinite loops or rabbit holes in autonomous research?
4. Design the control flow: centralized orchestrator vs decentralized agents?

Part B: Tool Integration

5. What tools/APIs would each agent need? (web search, PDF parsing, database queries, etc.)
6. How do you handle tool failures and retries?
7. Design error handling: what happens when an API rate limit is hit mid-research?
8. How do you validate tool outputs before passing to next agent?

Part C: Quality & Coherence

9. How do you ensure the final report is coherent despite being generated by multiple agents?
10. Design a fact-checking mechanism for cross-agent information.
11. How do you prevent contradiction in the final output?
12. Create a citation management system across agents.

Part D: Optimization & Control

13. How do you stay within the 10-15 minute time budget?
14. Design cost controls (some queries might trigger 100+ LLM calls).
15. How do you provide progress updates to the user?
16. Create a mechanism for users to intervene/guide mid-research.

Expected Answer Should Demonstrate:

Multi-Agent Architecture:

Orchestrator Agent (GPT-4)

↓

- └─> Query Decomposition Agent
 - Breaks research question into sub-questions
 - Identifies key concepts and relationships
- └─> Search Agent (parallel execution)
 - Academic papers (Semantic Scholar, arXiv)
 - News (web search API)
 - Patents (Google Patents)
 - Identifies 50-100 relevant sources
- └─> Filtering & Ranking Agent
 - Scores relevance of each source
 - Selects top 20-30 for deep reading
 - Balances recency, citation count, relevance
- └─> Reading & Extraction Agents (parallel)
 - 5-10 parallel agents each read 2-4 papers
 - Extract key findings, methods, results
 - Identify citations and connections
- └─> Synthesis Agent
 - Combines findings from reading agents
 - Identifies themes and patterns
 - Resolves contradictions
- └─> Network Analysis Agent
 - Maps researchers and institutions
 - Identifies collaboration patterns
 - Tracks citation networks
- └─> Report Generation Agent
 - Creates structured report
 - Ensures proper citations
 - Suggests future directions

Reasoning Strategy Selection:

- **ReAct:** For tool-use heavy agents (Search Agent, Reading Agent)

Thought: I need to find recent papers on quantum computing

Action: `search_semantic_scholar(query="quantum computing drug discovery", year`

Observation: Found 47 papers

Thought: Now I should filter for high-impact venues

Action: `filter_papers(min_citations=10, venues=["Nature", "Science"])`

- **Chain-of-Thought:** For synthesis and reasoning (Synthesis Agent)

Let's analyze the findings step by step:

1. Paper A found X with method M1
2. Paper B found Y with method M2
3. Methods M1 and M2 are complementary because...

Therefore, the field is moving toward...

- **Tree of Thoughts:** For complex planning (Orchestrator)

Research strategy options:

Branch 1: Breadth-first (many papers, surface-level)

Branch 2: Depth-first (fewer papers, detailed analysis)

Branch 3: Citation-guided (follow citation chains)

Evaluate each branch...

Select optimal: Branch 3 with depth-first on key papers

Safety & Control:

1. Loop Prevention:

```
class AgentOrchestrator:
    def __init__(self):
        self.max_iterations = 50
        self.max_tool_calls_per_agent = 20
        self.visited_states = set()
        self.timeout = 900 # 15 minutes

    def execute_agent(self, agent, task):
        state_hash = hash((agent.name, task))

        if state_hash in self.visited_states:
            return "Detected loop, skipping"

        self.visited_states.add(state_hash)

        with timeout(self.timeout):
            result = agent.execute(task)

        return result
```

2. Budget Control:

```
class CostController:
    def __init__(self, max_budget_usd=5.0):
```

```

self.max_budget = max_budget_usd
self.spent = 0
self.token_costs = {
    "gpt-4": {"input": 0.03/1000, "output": 0.06/1000},
    "gpt-3.5": {"input": 0.001/1000, "output": 0.002/1000}
}

def estimate_cost(self, model, input_tokens, output_tokens):
    cost = (input_tokens * self.token_costs[model]["input"] +
            output_tokens * self.token_costs[model]["output"])
    return cost

def approve_call(self, model, estimated_tokens):
    estimated_cost = self.estimate_cost(model, estimated_tokens, estimated_to

    if self.spent + estimated_cost > self.max_budget:
        # Switch to cheaper model or abort
        if model == "gpt-4":
            return "gpt-3.5", True # Downgrade
        else:
            return None, False # Abort

    return model, True

```

3. Progress Tracking:

```

class ProgressTracker:
    stages = [
        "Query decomposition",
        "Source search",
        "Source filtering",
        "Document reading",
        "Information synthesis",
        "Report generation"
    ]

    def update(self, stage, progress_pct, details):
        emit_event({
            "stage": stage,
            "progress": progress_pct,
            "details": details,
            "elapsed_time": time.time() - self.start_time
        })

```

Tool Integration:

Tool Registry:

```
tools = {
    "web_search": WebSearchTool(api_key=...),
    "semantic_scholar": SemanticScholarAPI(),
    "arxiv": ArxivAPI(),
    "pdf_reader": PDFExtractionTool(),
    "citation_graph": CitationNetworkTool(),
    "summarizer": SummarizationTool(model="gpt-3.5")
}
```

Error Handling:

```
class RobustToolExecutor:
    def execute(self, tool_name, params, max_retries=3):
        for attempt in range(max_retries):
            try:
                result = self.tools[tool_name].execute(**params)
                return {"success": True, "data": result}

            except RateLimitError as e:
                if attempt < max_retries - 1:
                    sleep_time = exponential_backoff(attempt)
                    time.sleep(sleep_time)
                else:
                    return {"success": False, "error": "Rate limit exceeded"}

            except ToolError as e:
                # Try fallback tool
                fallback = self.get_fallback_tool(tool_name)
                if fallback:
                    return self.execute(fallback, params, max_retries=1)
                else:
                    return {"success": False, "error": str(e)}
```

Quality Assurance:

1. Citation Management:

```
class CitationManager:
    def __init__(self):
        self.sources = {} # source_id -> full metadata
        self.claims = {} # claim -> [source_ids]

    def add_source(self, source_id, metadata):
```

```

        self.sources[source_id] = metadata

    def cite(self, claim, source_id):
        if source_id not in self.sources:
            raise ValueError(f"Source {source_id} not registered")

        if claim not in self.claims:
            self.claims[claim] = []
        self.claims[claim].append(source_id)

    def generate_bibliography(self):
        return [
            f"[{i+1}] {meta['authors']} ({meta['year']}). {meta['title']}. {meta['url']}"
            for i, meta in enumerate(self.sources.values())
        ]

```

2. Fact Checking:

```

class InterAgentValidator:
    def validate_claim(self, claim, supporting_agents):
        # Multiple agents should agree on facts
        agreements = []

        for agent in supporting_agents:
            verification = agent.verify_claim(claim)
            agreements.append(verification)

        consensus_score = sum(agreements) / len(agreements)

        if consensus_score < 0.7:
            return {"validated": False, "reason": "Low consensus"}

        return {"validated": True, "confidence": consensus_score}

```

3. Contradiction Detection:

```

def detect_contradictions(statements):
    # Use NLI model to check for contradictions
    for i, stmt1 in enumerate(statements):
        for stmt2 in statements[i+1:]:
            nli_result = nli_model(premise=stmt1, hypothesis=stmt2)

            if nli_result == "contradiction":
                yield {
                    "statement1": stmt1,
                    "statement2": stmt2,
                }

```

```

        "conflict": True
    }

```

Time Optimization:

```

# Parallel execution where possible
async def parallel_reading(papers, num_agents=10):
    reading_tasks = []

    for paper in papers:
        task = asyncio.create_task(
            reading_agent.analyze(paper)
        )
        reading_tasks.append(task)

    results = await asyncio.gather(*reading_tasks)
    return results

# Adaptive timeout based on remaining budget
def adaptive_timeout(stage, remaining_time):
    time_allocations = {
        "search": 0.15,      # 15% of time
        "filtering": 0.10,   # 10%
        "reading": 0.50,     # 50%
        "synthesis": 0.20,   # 20%
        "report": 0.05       # 5%
    }

    return remaining_time * time_allocations[stage]

```

User Interaction:

```

class InteractiveResearch:
    def execute(self, query):
        # Initial plan
        plan = self.orchestrator.create_plan(query)
        user_approved = self.present_plan_for_approval(plan)

        if not user_approved:
            plan = self.revise_plan(plan, user_feedback)

        # Execute with checkpoints
        for stage in plan.stages:
            result = self.execute_stage(stage)

        # Allow user to adjust

```

```
        if stage.allow_intervention:
            user_action = self.check_user_input()

            if user_action == "stop":
                return self.generate_partial_report()
            elif user_action == "adjust":
                stage = self.adjust_stage(stage, user_input)

    return self.generate_final_report()
```

Evaluation Rubric for All 5 Questions

Senior Level Candidate Should:

- Demonstrate system-level thinking, not just component solutions
- Consider trade-offs explicitly (cost, latency, accuracy, complexity)
- Discuss production concerns (monitoring, errors, scale)
- Provide specific technical details (algorithms, parameters, tools)
- Show awareness of latest research and tools
- Consider user experience and business impact
- Think about edge cases and failure modes
- Propose measurable success criteria

Red Flags:

- Single-solution thinking without considering alternatives
- Ignoring cost or latency